IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Marek Wisniewski
04/03/2005

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
- Summary of all results

Introduction

- Project background and context
- Problems you want to find answers

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Data was collected using SpaceX API calls to retrieve structured JSON data and web scraping to extract additional information from web pages, ensuring a scraping to extract additional information from web pages, ensuring a comprehensive dataset for analysis.
- Perform data wrangling
  - The data was processed by calculating launches per site analysing orbit and mission outcomes, creating a landing outcome label, determining the success rate and exporting the results to a CSV file
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash

Methodology

- Perform predictive analysis using classification models
  - Classification models (Logistic Regression, SVM, Decision Trees, KNN) were built and tested by standardizing the data, splitting into training and test sets, tuning hyperparameters using GridSearchCV, evaluating accuracy and confusion matrices, and selecting the best- performing model.

Data Collection

How data sets were collected?

The datasets were collected using two main methods: web scraping and API integration. For web scraping, the BeautifulSoup library was used to extract data from websites by parsing HTML, while APIs provided structured data through direct requests to their endpoints. After collection, the data was cleaned, processed, and organized into structured datasets for further analysis

Data Collection – SpaceX API

1.Requesting rocket launch data from SpaceX API:

```
spacex_url="https://api.spacexdata.com/v4/launches/past"

response = requests.get(spacex_url)
```

2.Converting Response to a JSON file:

```
10]: response=requests.get(static_json_url)
```

3.Using custom functions to clean data:

```
[34]: # Call getBoosterVersion
      getBoosterVersion(data)

[ ]: # Call getLaunchSite
     getLaunchSite(data)

[ ]: # Call getPayloadData
     getPayloadData(data)

[ ]: # Call getCoreData
     getCoreData(data)
```

4.Combining the columns into a dictionary to create data frame:

```
launch_dict = {'FlightNumber': list(data['flight_number']),
'Date': list(data['date']),
'BoosterVersion':BoosterVersion,
'PayloadMass':PayloadMass,
'Orbit':Orbit,
'LaunchSite':LaunchSite,
'Outcome':Outcome,
'Flights':Flights,
'GridFins':GridFins,
'Reused':Reused,
'Legs':Legs,
'LandingPad':LandingPad,
'Block':Block,
'ReusedCount':ReusedCount,
'Serial':Serial,
'Longitude': Longitude,
'Latitude': Latitude}
```

5.Filtering dataframe and exporting to a CSV:

```
[25]: data_falcon9 = df[df['BoosterVersion'] == 'Falcon 9']

[ ]: data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

The GitHub URL

Data Collection - Scraping

1.Request the Falcon9 Launch Wiki page from its URL:

```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
response_ = requests.get(static_url)
```

2. Extract all column/variable names from the HTML table header:

```
html_tables = soup.find_all('table')
```

3.Create a data frame by parsing the launch HTML tables:

```
launch_dict= dict.fromkeys(column_names)


del launch_dict['Date and time ( )']


launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]
```

4.Create a dataframe from launch_dict :

```
df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

5. Export dataframe to a CSV:

```
df.to_csv('spacex_web_scraped.csv', index=False)
```

The GitHub URL.

Data Wrangling

How data were processed: The percentage of missing values in each attribute was calculated to assess data completeness, and columns were identified as either numerical or categorical to guide appropriate analysis and modeling techniques.

Data wrangling process:

- Calculate the number of launches on each site:

```
df['LaunchSite'].value_counts()
```

- Calculate the number and occurrence of each orbit:

```
df['Orbit'].value_counts()
```

- Calculate the number and occurence of mission outcome of the orbits:

```
landing_outcomes = df['Outcome'].value_counts()
```

- Create a landing outcome label from Outcome column:

```
landing_class = df['Outcome'].apply(lambda x:0 if x in bad_outcomes else 1).tolist()
```

- export dataframe to a CSV:

```
df.to_csv("dataset_part_2.csv", index=False)
```

The GitHub URL.

EDA with Data Visualization

I used scatter, bar, and line charts because they allow for easy visualization of relationships, trends, and comparisons in the data, which helps in identifying patterns and insights during exploratory data analysis.

The GitHub URL.

EDA with SQL

- Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT LAUNCH_SITE FROM SPACEXTBL
```

- Display 5 records where launch sites begin with the string 'CCA'

```
%sql SELECT LAUNCH_SITE FROM SPACEXTBL WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5
```

- Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql SELECT sum(payload_mass__kg_) as total_payload FROM SPACEXTBL WHERE customer LIKE 'NASA (CRS)%';
```

- Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG(payload_mass__kg_) FROM SPACEXTBL WHERE booster_version = 'F9 v1.1'
```

- List the date when the first succesful landing outcome in ground pad was acheived.

```
%sql SELECT MIN(DATE) FROM SPACEXTBL WHERE Landing_Outcome = 'Success (ground pad)'
```

- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000 AND Landing_Outcome = 'Success (drone ship)';
```

EDA with SQL

- List the total number of successful and failure mission outcomes

```
%sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000 AND Landing_Outcome = 'Success (drone ship)';
```

- List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
sql SELECT MISSION_OUTCOME, COUNT(*) AS TOTAL FROM SPACEXTBL GROUP BY MISSION_OUTCOME ORDER BY MISSION_OUTCOME;
```

- List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL) ORDER BY BOOSTER_VERSION;
```

- List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

```
%%sql SELECT "Booster_Version", "Launch_Site" FROM SPACEXTABLE

WHERE "Landing_Outcome" = 'Failure (drone ship)' AND substr(Date,1,4) = '2015';
```

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%%sql SELECT "LANDING_OUTCOME", COUNT(*) as 'COUNT' FROM SPACEXTBL

WHERE substr(Date,1,4) || substr(Date,6,2) || substr(Date,9,2)

between '20100604' and '20170320' GROUP BY "Landing_Outcome" ORDER BY "COUNT" DESC;
```

The GitHub URL.

Build an Interactive Map with Folium

- Summarize what map objects such as markers, circles, lines, etc. you created and added to a folium map:
  - A blue circle was created at the NASA Johnson Space Center's coordinates with a popup label showing its name.
  - A blue circle was created at the NASA Johnson Space Center's coordinates with an icon showing its name.
  - A folium.Circle and folium.Marker were created and added for each launch site on the site map.
  - Markers were created for all launch records.
  - For each launch result in the spacex_df dataframe, a folium.Marker was added to the marker_cluster.
  - The MousePosition feature was added to the map.
  - A PolyLine was drawn between a launch site and the selected coastline point.
  - A PolyLine was drawn between a launch site and the railways point.
- Explain why you added those objects

  I used these objects to visualize data interactively, analyze the relationships between location and mission success, and identify optimal locations for future launch sites based on existing data.
- **The GitHub URL.**

- ## Summarize what plots/graphs and interactions you have added to a dashboard

- The following elements were added to the dashboard: a pie chart, a range slider, a dropdown menu, and a scatter plot. These were included to enhance data visualization, enable interactive filtering, and provide deeper insights into the dataset.
- These interactions and plots were included to make the dashboard more user-friendly, interactive, and insightful for exploring the dataset effectively.

- ## Explain why you added those plots and interactions:

- These interactions and plots were included to make the dashboard more user-friendly, interactive, and insightful for exploring the dataset effectively.

The GitHub URL.

Predictive Analysis (Classification)

- **Summarize how you built, evaluated, improved, and found the best performing classification model**

Classification models (Logistic Regression, SVM, Decision Trees, KNN) were built and tested by standardizing the data, splitting into training and test sets, tuning hyperparameters using GridSearchCV, evaluating accuracy and confusion matrices, and selecting the best-performing model.

- **Model development process using key phrases and flowchart**

- Standardize the data in X then reassign it to the variable X using the transform provided below.

```
transform = preprocessing.StandardScaler()
X = transform.fit_transform(X)
```

- Use the function train_test_split to split the data X and Y into training and test data.

```
X_train, X_test, Y_train, Y_test = train_test_split( X, Y, test_size=0.2, random_state=2)
```

- Create a logistic regression object then create a GridSearchCV object logreg_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
parameters ={'C':[0.01,0.1,1],
             'penalty':['l2'],
             'solver':['lbfgs']}
```

```
parameters ={"C":[0.01,0.1,1],'penalty':['l2'], 'solver':['lbfgs']}# l1 lasso l2 ridge
lr=LogisticRegression()
logreg_cv=GridSearchCV(lr, parameters,cv=10)
logreg_cv.fit(X_train,Y_train)
```

Predictive Analysis (Classification)

- Calculate the accuracy on the test data using the method score:

```
logreg_cv.score(X_test,Y_test)
```

- Lets look at the confusion matrix:

```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

- Create a support vector machine object then create a GridSearchCV object svm_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
```

```
gs1 = GridSearchCV(svm, parameters, scoring='accuracy', cv=10)

svm_cv = gs1.fit(X_train, Y_train)
```

```
print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)
```

Predictive Analysis (Classification)

- Calculate the accuracy on the test data using the method score:

```
svm_cv.score(X_train, Y_train)
```

- We can plot the confusion matrix

```
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

- Create a decision tree classifier object then create a GridSearchCV object tree_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
parameters = {'criterion': ['gini', 'entropy'],
     'splitter': ['best', 'random'],
     'max_depth': [2*n for n in range(1,10)],
     'max_features': ['log2', 'sqrt'],
     'min_samples_leaf': [1, 2, 4],
     'min_samples_split': [2, 5, 10]}

tree = DecisionTreeClassifier()
```

```
grid_search_dt = GridSearchCV(tree, parameters,cv=10)
tree_cv = grid_search_dt.fit(X_train, Y_train)
```

```
print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

- Calculate the accuracy of tree_cv on the test data using the method score:

```
print('Accuracy on test data is: {:.3f}'.format(tree_cv.score(X_test, Y_test)))
```

Predictive Analysis (Classification)

- We can plot the confusion matrix

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

- Create a k nearest neighbors object then create a GridSearchCV object knn_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
              'p': [1,2]}

KNN = KNeighborsClassifier()
```

```
knn_cv = GridSearchCV(KNN, parameters, cv=10)

knn_cv.fit(X,Y)
```

- Calculate the accuracy of knn_cv on the test data using the method score:

```
print('Accuracy on test data is: {:.3f}'.format(knn_cv.score(X_test, Y_test)))
```

- We can plot the confusion matrix

```
yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

- **The GitHub URL.**

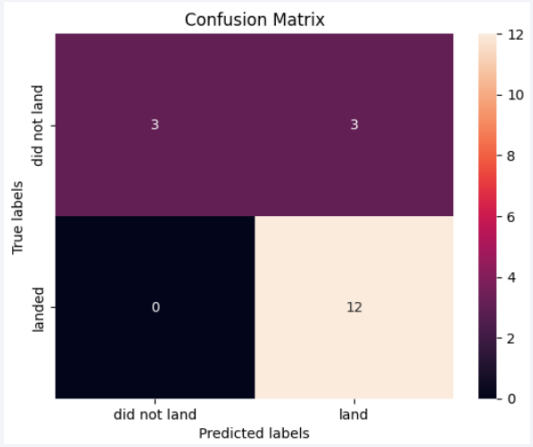
Results

Exploratory data analysis results

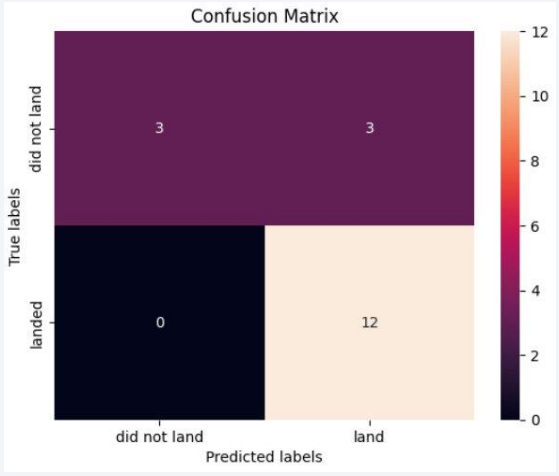
- Logistic Regression: Accuracy: 0.8333 (83.33%)
- Support Vector Machine (SVM): Accuracy: 0.8889 (88.89%)
- Decision Tree: Accuracy: 0.722 (72.2%)
- K-Nearest Neighbors (KNN): Accuracy: 0.944 (94.4%)

Interactive analytics demo in screenshots

logistic regression

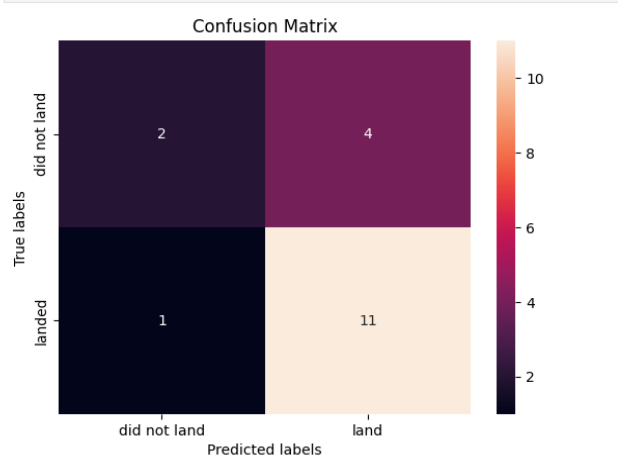


svm_cv

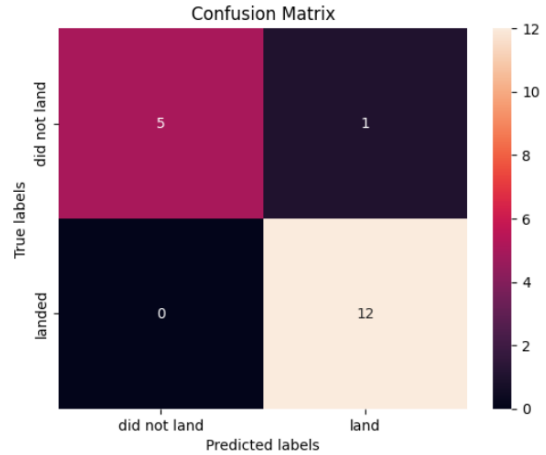


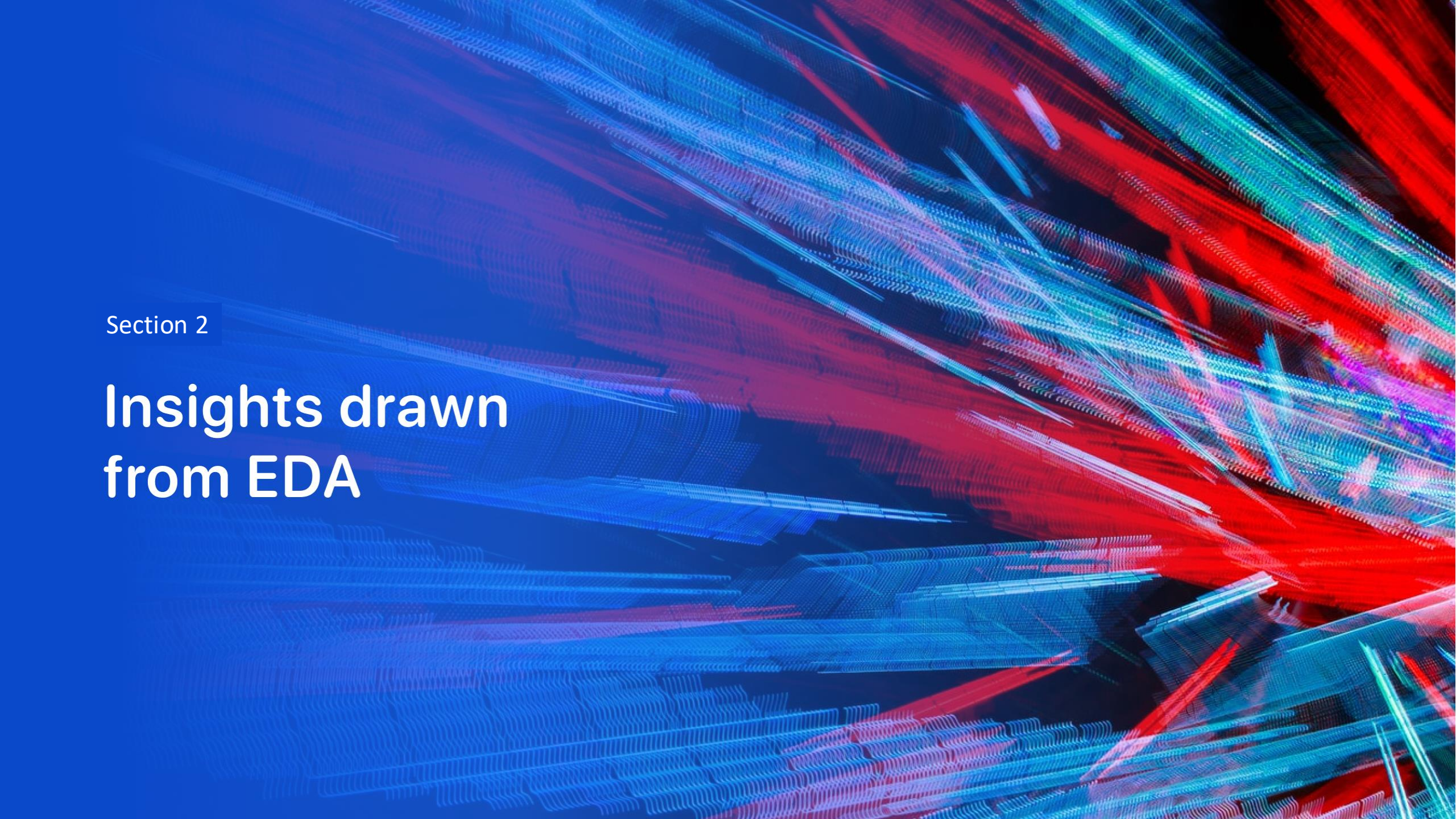
Results

tree_cv



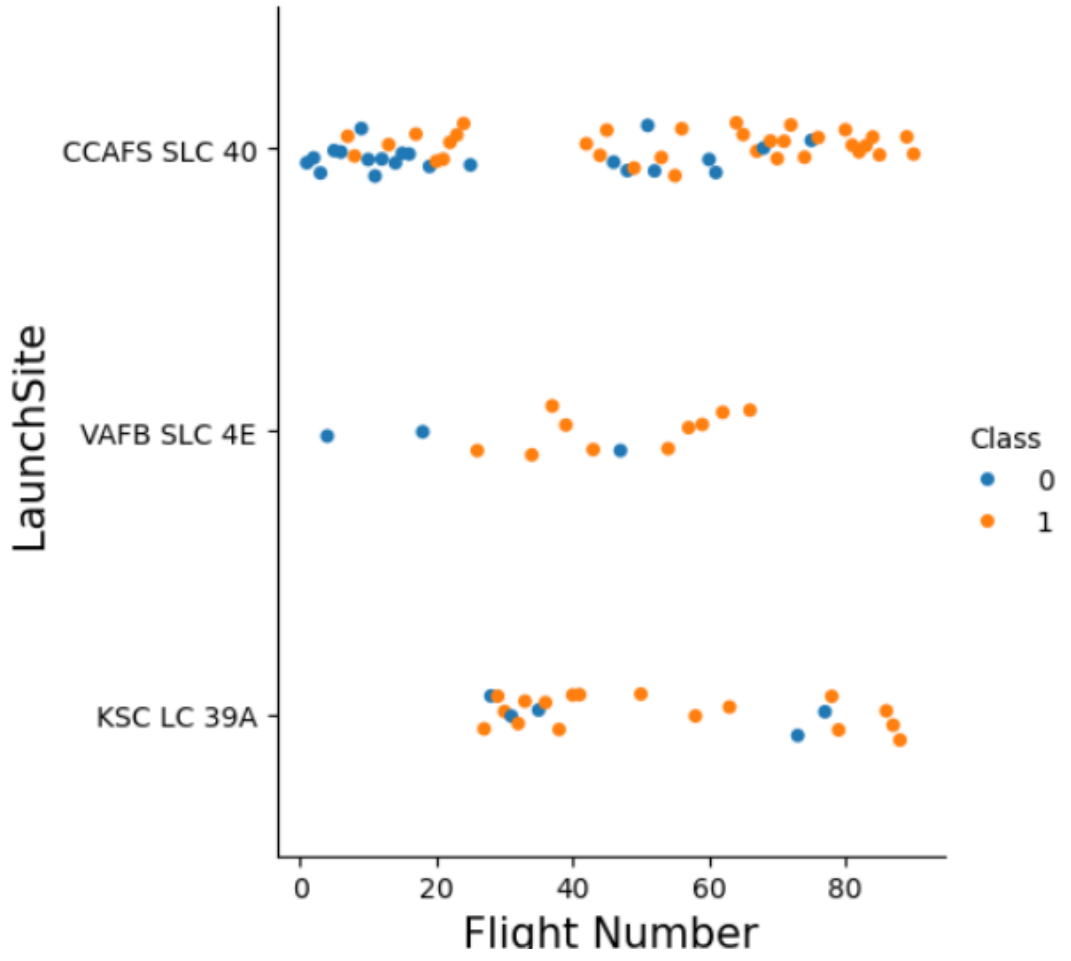
knn_cv




Section 2
Insights drawn from EDA

Flight Number vs. Launch Site

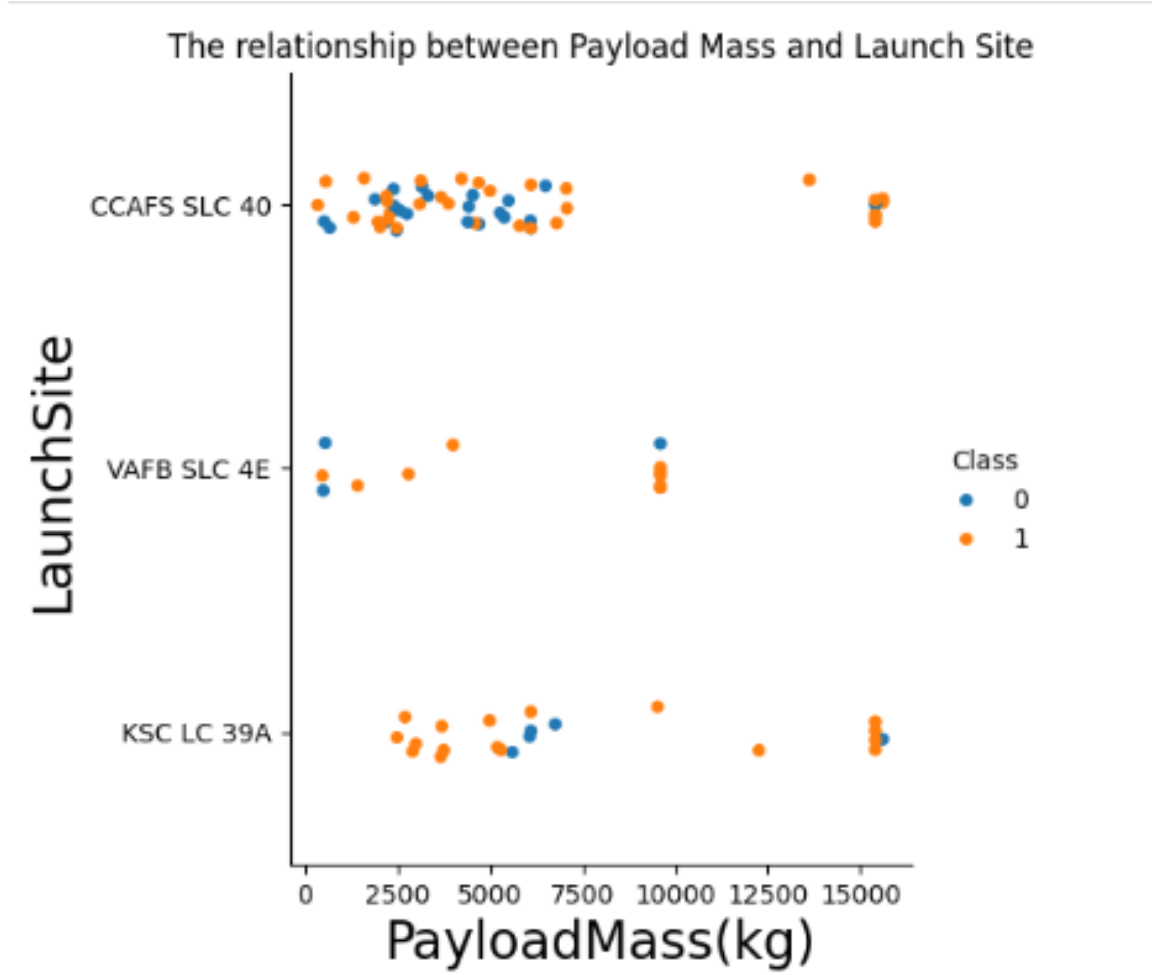
The scatter plot shows the relationship between flight number (Flight Number) and launch site (Launch Site), with classes (Class) divided into 0 (no landing) and 1 (landing).



Payload vs. Launch Site

The scatter plot shows the relationship between payload mass (Payload Mass) and launch site (Launch Site), with classes (Class) divided into 0 (no landing) and 1 (landing).

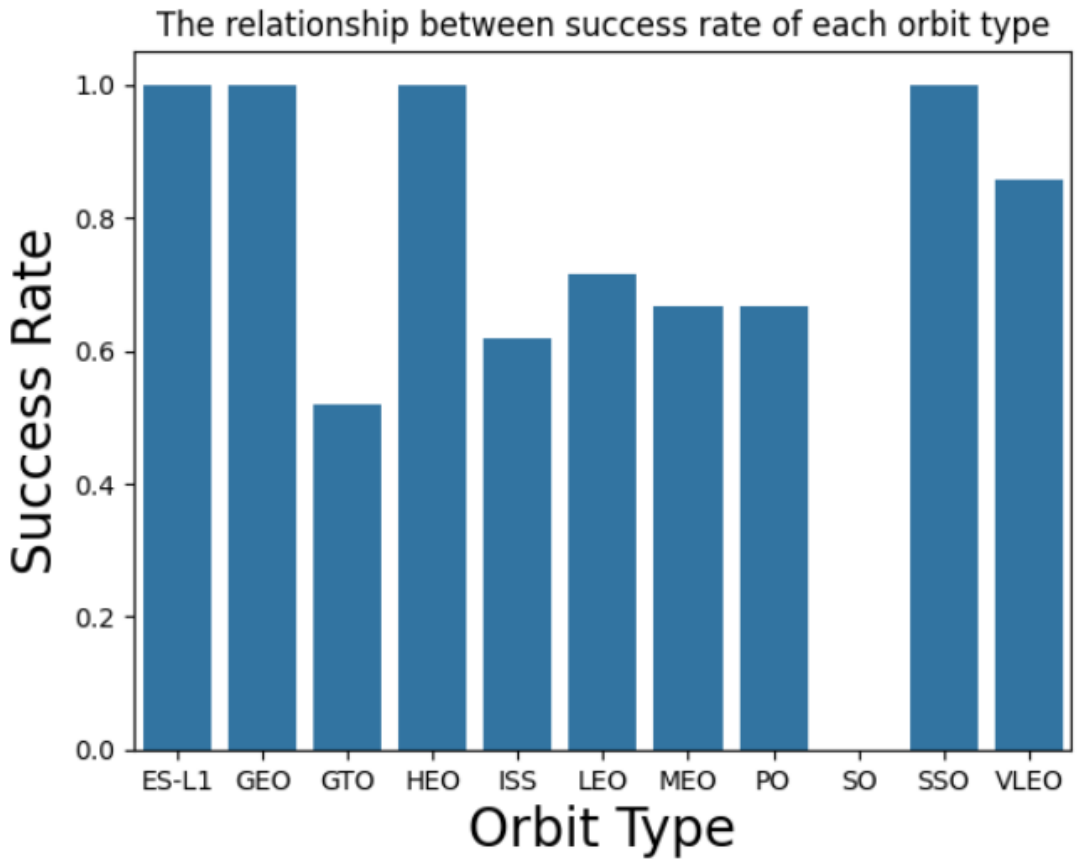
The chart allows for an analysis of how payload mass affects mission outcomes depending on the launch site.



Success Rate vs. Orbit Type

The bar chart shows the success rate for each orbit type.

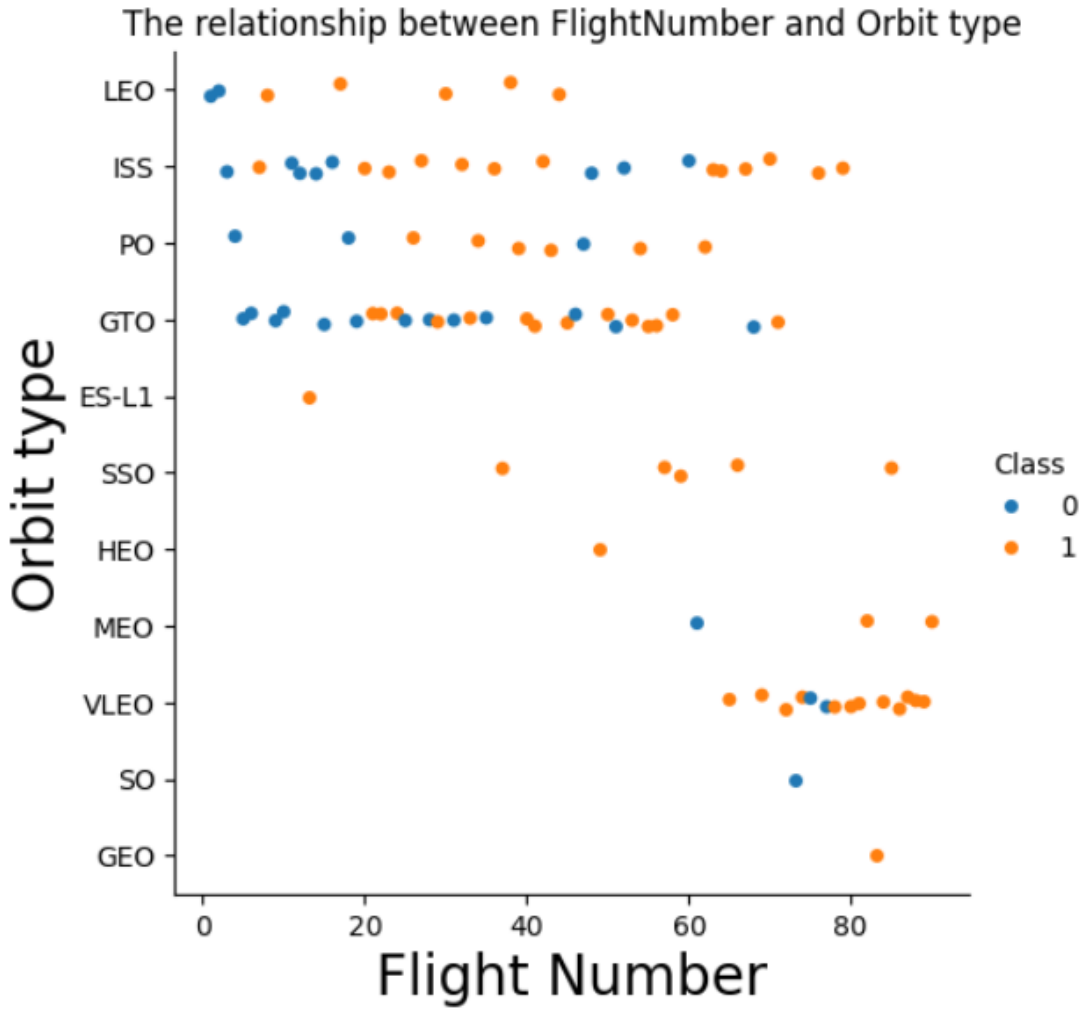
The chart allows for a clear comparison of which orbit types have the highest and lowest success rates.



Flight Number vs. Orbit Type

The scatter plot shows the relationship between flight number (Flight Number) and orbit type, with classes (Class) divided into 0 (no landing) and 1 (landing).

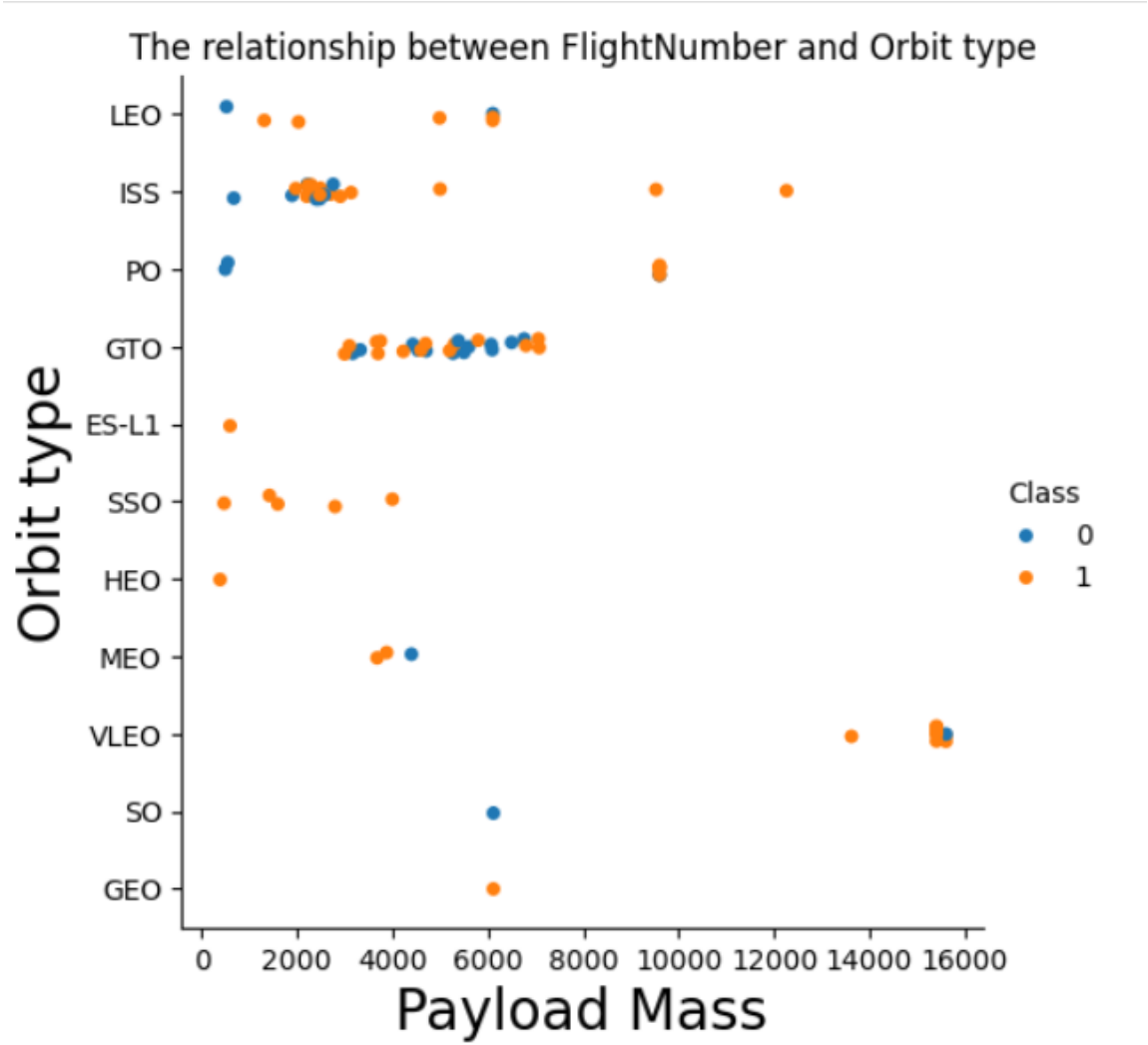
The chart allows for an analysis of how flight number and orbit type relate to mission success or failure.



Payload vs. Orbit Type

The scatter plot shows the relationship between payload mass (Payload Mass) and orbit type, with classes (Class) divided into 0 (no landing) and 1 (landing).

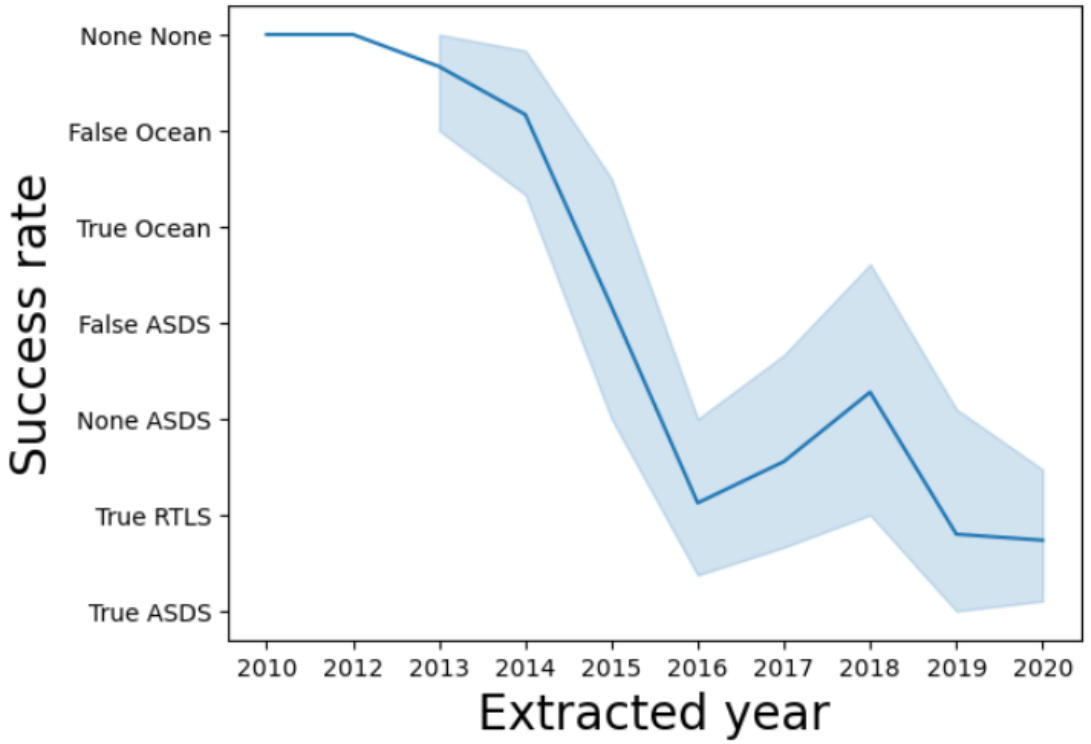
The chart allows for an analysis of how payload mass and orbit type relate to mission success or failure.



Launch Success vs Yearly Trend

The line chart shows the yearly trend of launch success rates, categorized by landing outcomes.

The chart allows for an analysis of how launch success rates have evolved over the years, depending on the landing method.



All Launch Site Names

This output provides a overview of the different launch locations used in the missions.

Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT LAUNCH_SITE FROM SPACEXTBL
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

The code displays 5 records where launch site names begin with the string 'CCA'.

This query helps identify launch sites with names starting with 'CCA'.

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%sql SELECT LAUNCH_SITE FROM SPACEXTBL WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40

Total Payload Mass

The code calculates the total payload mass carried by boosters launched for NASA (CRS missions).

This query provides insight into the total payload capacity utilized for NASA's CRS missions.

```
%sql SELECT sum(payload_mass__kg_) as total_payload FROM SPACEXTBL WHERE customer LIKE 'NASA (CRS)%';
```

```
 * sqlite:///my_data1.db
Done.
```

total_payload
48213

Average Payload Mass by F9 v1.1

The code calculates the average payload mass carried by the booster version F9 v1.1.

This query provides insight into the typical payload capacity of the F9 v1.1 booster version.

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG(payload_mass__kg_) FROM SPACEXTBL WHERE booster_version = 'F9 v1.1'
```

```
 * sqlite:///my_data1.db
Done.
```

AVG(payload_mass__kg_)
2928.4

First Successful Ground Landing Date

The code identifies the date of the first successful ground pad landing outcome.

```
%sql SELECT MIN(DATE) FROM SPACEXTBL WHERE Landing_Outcome = 'Success (ground pad)'
```

```
 * sqlite:///my_data1.db
Done.
```

MIN(DATE)
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

The code lists the names of boosters that successfully landed on a drone ship and carried a payload mass between 4,000 kg and 6,000 kg.

The result shows the distinct booster versions that met these criteria:

- F9 FT B1022
- F9 FT B1026
- F9 FT B1021.2
- F9 FT B1031.2

```
%sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000 AND Landing_Outcome = 'Success (drone ship)';
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

The code lists the total number of successful and failed mission outcomes.

This query provides a summary of mission success and failure rates.

```
sql SELECT MISSION_OUTCOME, COUNT(*) AS TOTAL FROM SPACEXTBL GROUP BY MISSION_OUTCOME ORDER BY MISSION_OUTCOME;
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	TOTAL
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

The code lists the names of booster versions that carried the maximum payload mass.

This query identifies the boosters that handled the heaviest payloads.

```
sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL) ORDER BY BOOSTER_VERSION;
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1048.5
F9 B5 B1049.4
F9 B5 B1049.5
F9 B5 B1049.7
F9 B5 B1051.3
F9 B5 B1051.4
F9 B5 B1051.6
F9 B5 B1056.4
F9 B5 B1058.3
F9 B5 B1060.2
F9 B5 B1060.3

2015 Launch Records

The code lists the 2015 launch records for boosters that experienced a failure during drone ship landing.

This query provides insight into missions from 2015 that did not achieve a successful drone ship landing.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%%sql SELECT "Booster_Version", "Launch_Site" FROM SPACEXTABLE

WHERE "Landing_Outcome" = 'Failure (drone ship)' AND substr(Date,1,4) = '2015';
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	Launch_Site
F9 v1.1 B1012	CCAFS LC-40
F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

The code ranks the count of landing outcomes between June 4, 2010, and March 20, 2017, in descending order.

This query provides a breakdown of landing outcomes during the specified period, highlighting the most and least frequent results.

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%%sql SELECT "LANDING_OUTCOME", COUNT(*) as 'COUNT' FROM SPACEXTBL

WHERE substr(Date,1,4) || substr(Date,6,2) || substr(Date,9,2)

between '20100604' and '20170320' GROUP BY "Landing_Outcome" ORDER BY "COUNT" DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

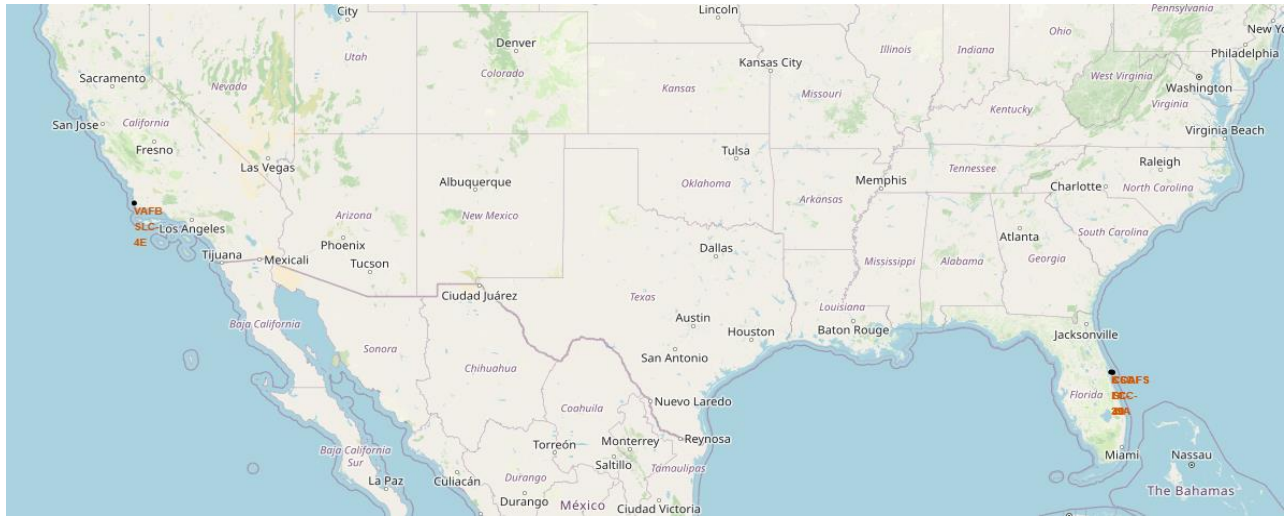
Landing_Outcome	COUNT
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Section 3

Launch Sites Proximities Analysis

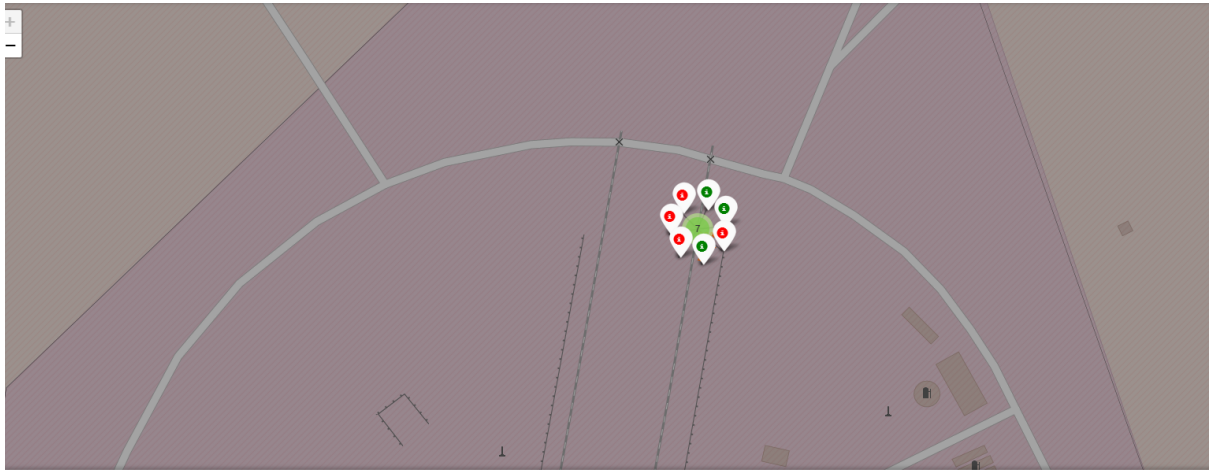
Map of All Launch Sites.

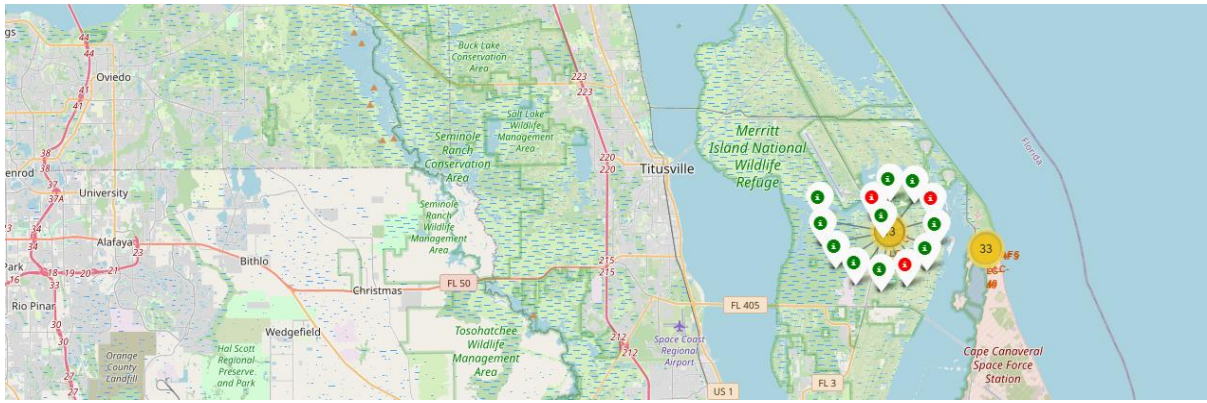
Each location is marked with a black circle and labeled in red font. The map has been designed in a clear manner to enable easy identification and localization of individual launch sites.

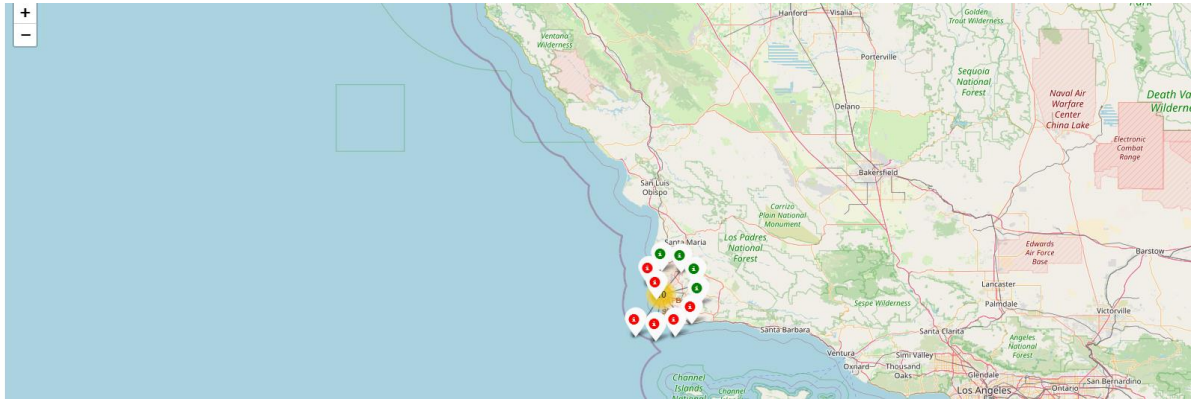


Maps of Success and Failed Launches by Site

A green circle indicates a successful launch, while a red one marks a failure. The map makes it easy to identify the outcomes of launches from each site.

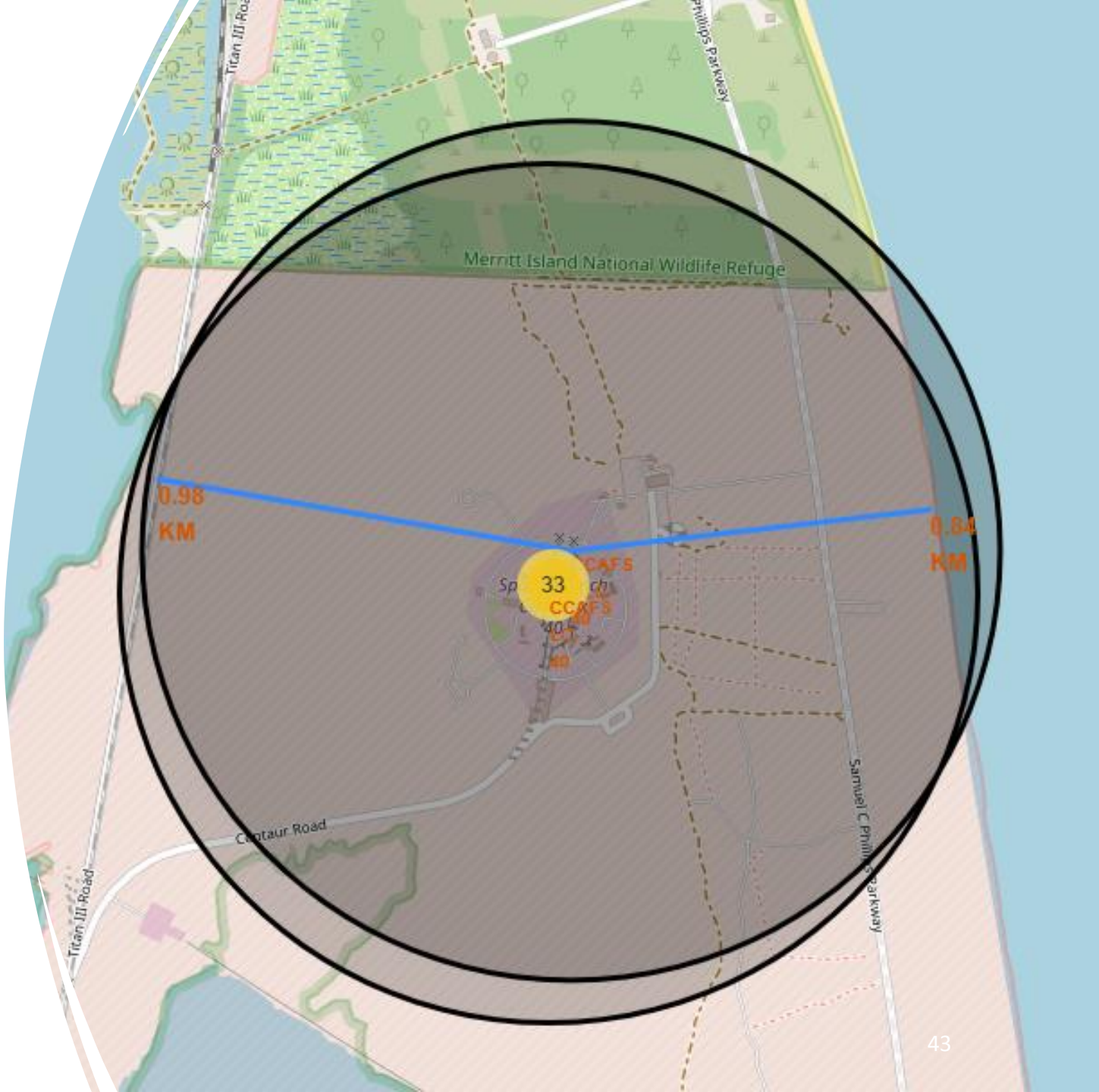


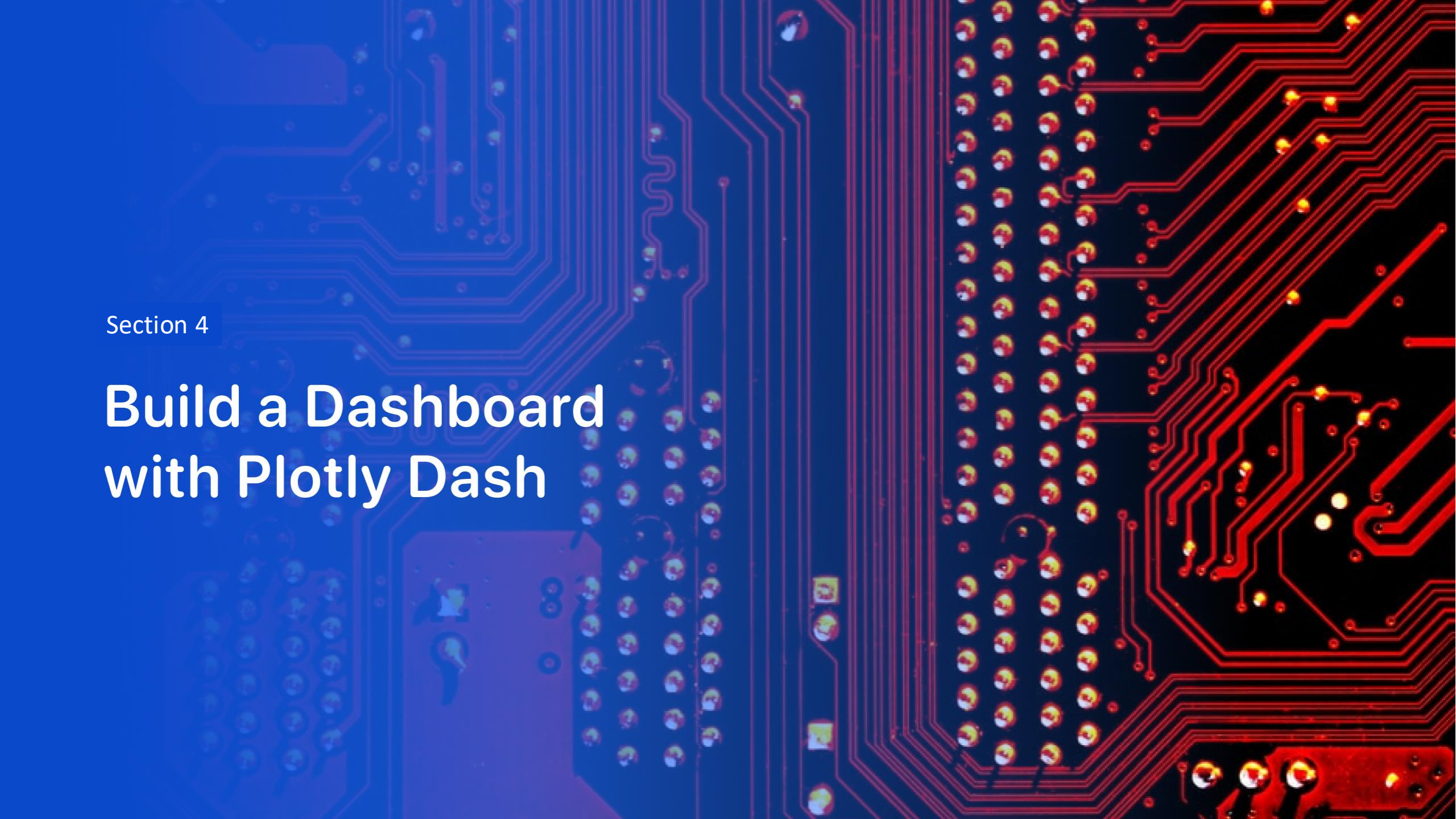




The PolyLine between a launch site to the railway.

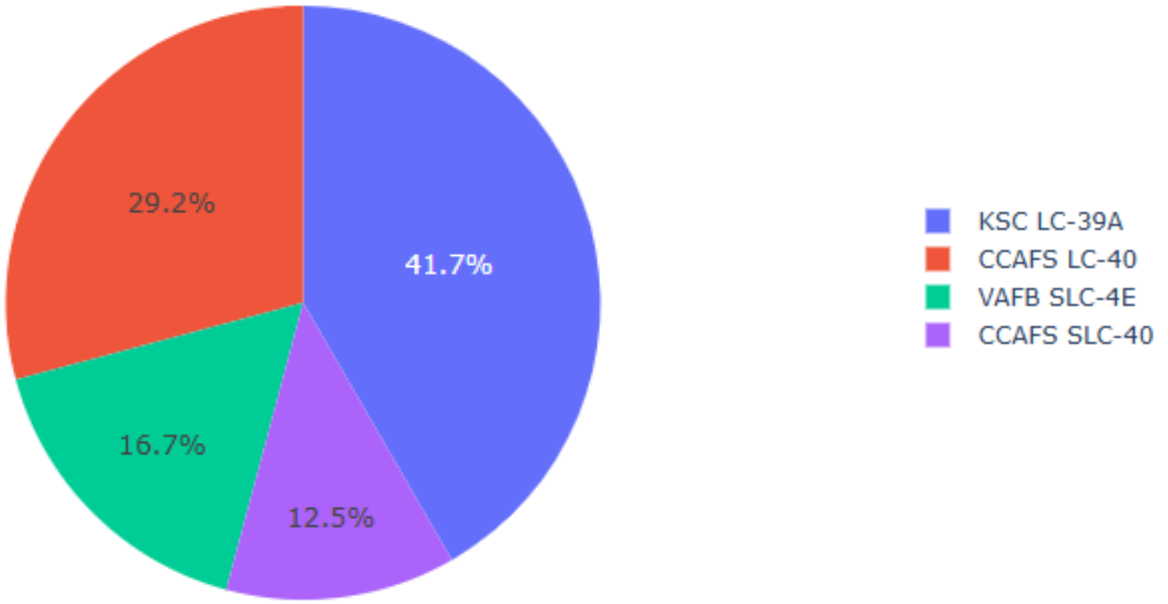
The blue line connects the launch site to the railway, and the distance between them is labeled in red font. The map allows for quick and clear reading of the distance between these points.




Section 4
Build a Dashboard with Plotly Dash

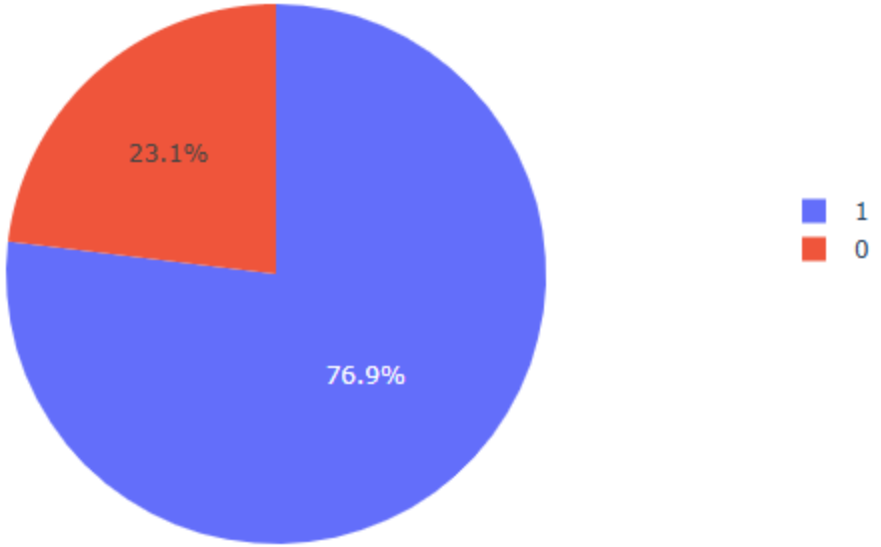
Piechart of Successful Launches per Site

Each color on the pie chart corresponds to a different location, and their names are explained in the legend. The chart shows the number of successful launches for each site.

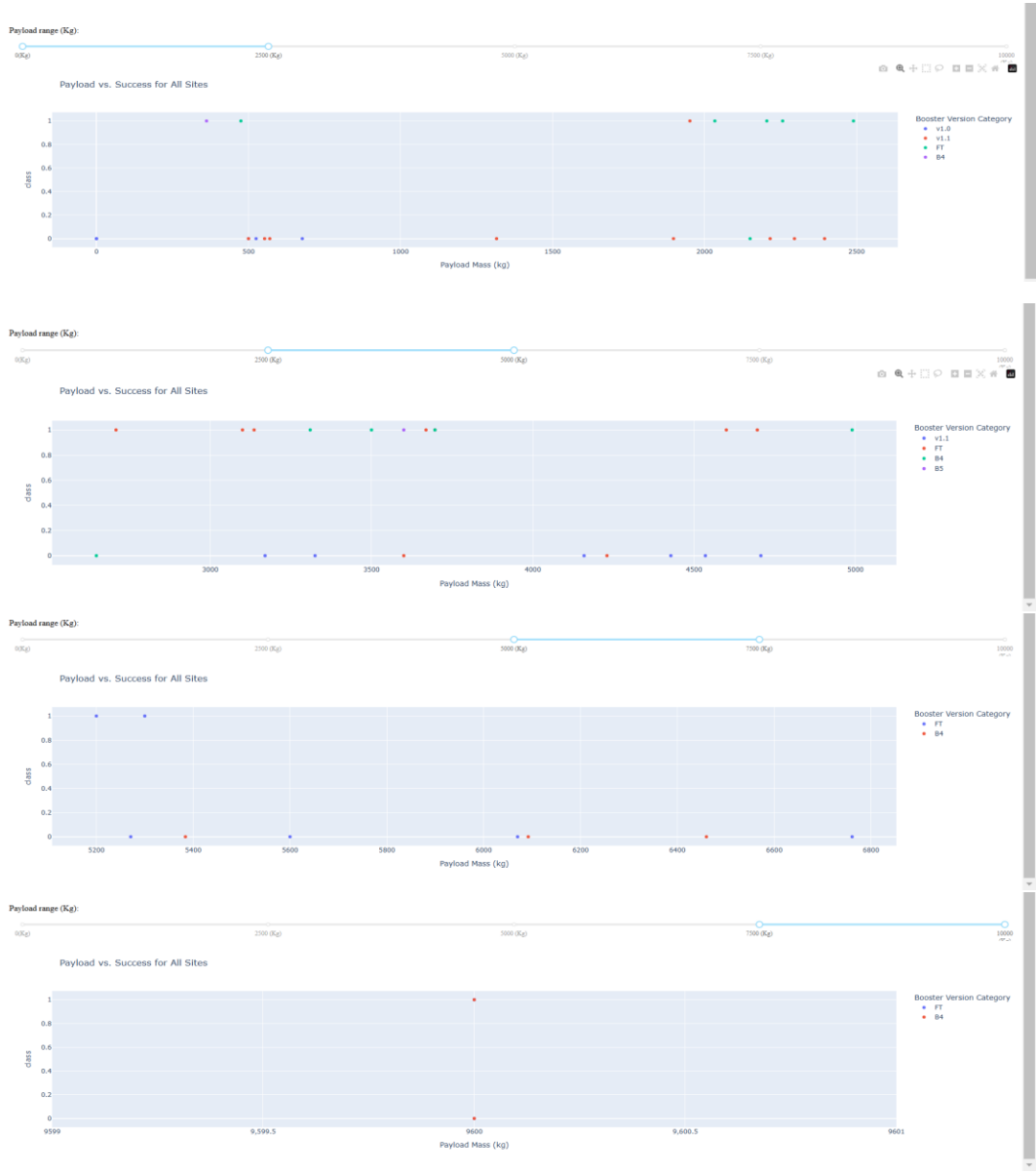


Piechart of the Launch Site with the Highest Success Ratio

The blue color on the pie chart represents successful launches, while the red color indicates failures. The chart shows the ratio of successes to failures for Site KSC LC-39A



Scatter Plot: Payload vs. Launch Outcome for All Sites (Interactive Range Slider)



The charts show the relationship between payload mass and success rate for four different ranges:

- 0–2500 kg
- 2500–5000 kg
- 5000–7500 kg
- 7500+ kg

Each color corresponds to a different booster version (e.g., YL, L, FT), allowing for a comparison of their effectiveness across different payload mass ranges.

The charts enable an analysis of which booster versions are most efficient depending on the payload mass.

Section 5

Predictive Analysis (Classification)

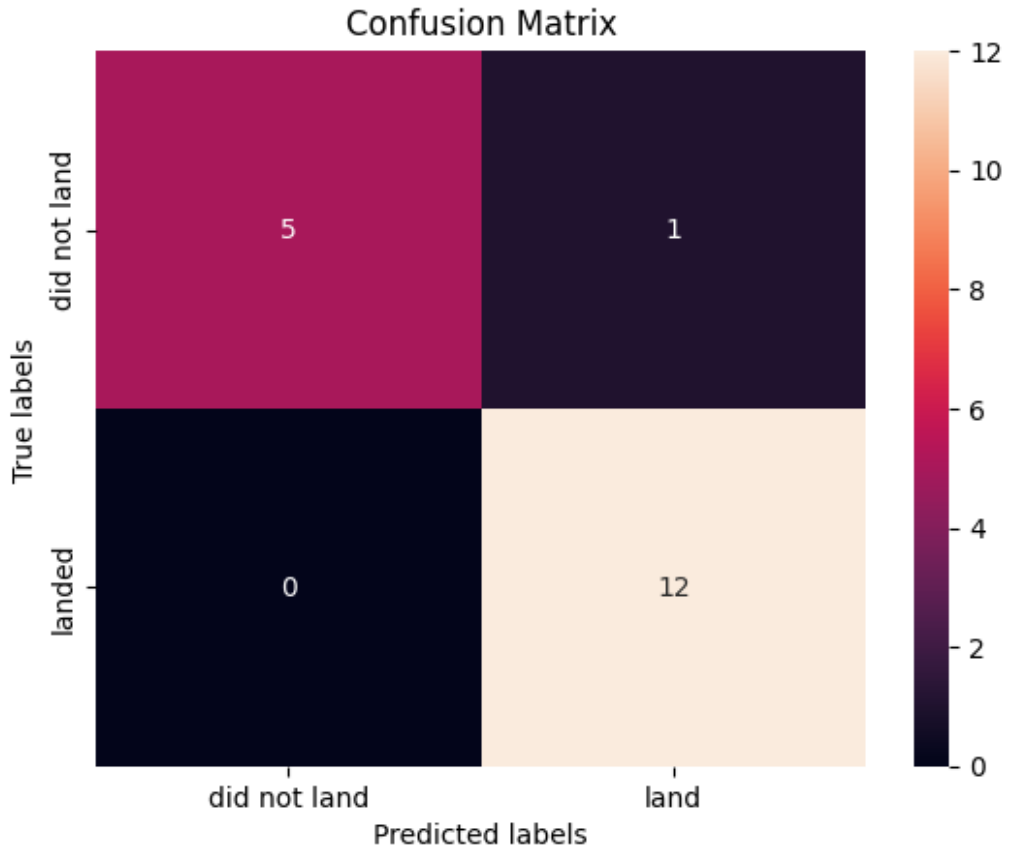
Classification Accuracy

The model with the highest classification accuracy (94.44%) is KNN (K-Nearest Neighbors).

```
[83.33333333333334, 88.88888888888889, 72.22222222222221, 94.44444444444444]
['LogReg', 'SVM', 'Tree', 'KNN']
```

Confusion Matrix

The confusion matrix shows how well the model predicts "landing" or "not landing". The model is good at predicting "landing", but has one error in predicting "not landing".



Conclusions

- **Mission Success Rate**: The majority of missions were successful (98 cases), while only one mission ended in failure during flight.
- **Landings**: The most common landing outcome was "no attempt" (10 cases), and the most frequent successful outcome was landing on a drone ship (5 cases).
- **Boosters**: The F9 B5 booster versions were most frequently used to carry maximum payloads, indicating their high efficiency.
- **Time Trends**: The first successful ground pad landing occurred on December 22, 2015, marking a milestone in reusable technology development.
- **Payloads**: The average payload mass for the F9 v1.1 booster was 2,928.4 kg, and the total payload mass for NASA (CRS) missions was 48,213 kg.
- **Orbits**: Missions to LEO and GTO orbits had the highest success rates, while orbits such as ES-L1 and HEO were less frequently used.

Thank you!